

```

code/opt/vec.c
1  data_t *get_vec_start(vec_ptr v)
2  {
3      return v->data;
4  }

code/opt/vec.c
1  /* Direct access to vector data */
2  void combine3(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t *data = get_vec_start(v);
7
8      *dest = IDENT;
9      for (i = 0; i < length; i++) {
10         *dest = *dest OP data[i];
11     }
12 }

```

图 5-9 消除循环中的函数调用。结果代码没有显示性能提升，但是它有其他的优化

令人吃惊的是，性能没有明显的提升。事实上，整数求和的性能还略有下降。显然，内循环中的其他操作形成了瓶颈，限制性能超过调用 `get_vec_element`。我们还会再回到这个函数(见 5.11.2 节)，看看为什么 `combine2` 中反复的边界检查不会让性能更差。而现在，我们可以将这个转换视为一系列步骤中的一步，这些步骤将最终产生显著的性能提升。

5.6 消除不必要的内存引用

`combine3` 的代码将合并运算计算的值累积在指针 `dest` 指定的位置。通过检查编译出来的为内循环产生的汇编代码，可以看出这个属性。在此我们给出数据类型为 `double`，合并运算为乘法的 x86-64 代码：

```

Inner loop of combine3. data_t = double, OP = *
dest in %rbx, data+i in %rdx, data+length in %rax
1  .L17:                                loop:
2      vmovsd (%rbx), %xmm0             Read product from dest
3      vmulsd (%rdx), %xmm0, %xmm0      Multiply product by data[i]
4      vmovsd %xmm0, (%rbx)            Store product at dest
5      addq $8, %rdx                   Increment data+i
6      cmpq %rax, %rdx                 Compare to data+length
7      jne .L17                        If !=, goto loop

```

在这段循环代码中，我们看到，指针 `dest` 的地址存放在寄存器 `%rbx` 中，它还改变了代码，将第 `i` 个数据元素的指针保存在寄存器 `%rdx` 中，注释中显示为 `data+i`。每次迭代，这个指针都加 8。循环终止操作通过比较这个指针与保存在寄存器 `%rax` 中的数值来判断。我们可以看到每次迭代时，累积变量的数值都要从内存读出再写入到内存。这样的读写很浪费，因为每次迭代开始时从 `dest` 读出的值就是上次迭代最后写入的值。

我们能够消除这种不必要的内存读写，按照图 5-10 中 `combine4` 所示的方式重写代码。引入一个临时变量 `acc`，它在循环中用来累积计算出来的值。只有在循环完成之后结果才存放在 `dest` 中。正如下面的汇编代码所示，编译器现在可以用寄存器 `%xmm0` 来保存

累积值。与 combine3 中的循环相比，我们将每次迭代的内存操作从两次读和一次写减少到只需要一次读。

```

Inner loop of combine4. data_t = double, OP = *
acc in %xmm0, data+i in %rdx, data+length in %rax
1  .L25:                                loop:
2    vmulsd  (%rdx), %xmm0, %xmm0      Multiply acc by data[i]
3    addq    $8, %rdx                  Increment data+i
4    cmpq    %rax, %rdx                Compare to data+length
5    jne     .L25                      If !=, goto loop

```

```

1  /* Accumulate result in local variable */
2  void combine4(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t *data = get_vec_start(v);
7      data_t acc = IDENT;
8
9      for (i = 0; i < length; i++) {
10         acc = acc OP data[i];
11     }
12     *dest = acc;
13 }

```

图 5-10 把结果累积在临时变量中。将累积值存放在局部变量 acc(累积器(accumulator)的简写)中，消除了每次循环迭代中从内存中读出并将更新值写回的需要

我们看到程序性能有了显著的提高，如下表所示：

函数	方法	整数		浮点数	
		+	*	+	*
combine3	直接数据访问	7.17	9.02	9.02	11.03
combine4	累积在临时变量中	1.27	3.01	3.01	5.01

所有的时间改进范围从 $2.2\times$ 到 $5.7\times$ ，整数加法情况的时间下降到了每元素只需 1.27 个时钟周期。

可能又有人会认为编译器应该能够自动将图 5-9 中所示的 combine3 的代码转换为在寄存器中累积那个值，就像图 5-10 中所示的 combine4 的代码所做的那样。然而实际上，由于内存别名使用，两个函数可能会有不同的行为。例如，考虑整数数据，运算为乘法，标识元素为 1 的情况。设 $v=[2, 3, 5]$ 是一个由 3 个元素组成的向量，考虑下面两个函数调用：

```

combine3(v, get_vec_start(v) + 2);
combine4(v, get_vec_start(v) + 2);

```

也就是在向量最后一个元素和存放结果的目标之间创建一个别名。那么，这两个函数的执行如下：

函数	初始值	循环之前	i = 0	i = 1	i = 2	最后
combine3	[2, 3, 5]	[2, 3, 1]	[2, 3, 2]	[2, 3, 6]	[2, 3, 36]	[2, 3, 36]
combine4	[2, 3, 5]	[2, 3, 5]	[2, 3, 5]	[2, 3, 5]	[2, 3, 5]	[2, 3, 30]

正如前面讲到过的，combine3 将它的结果累积在目标位置中，在本例中，目标位置就是向量的最后一个元素。因此，这个值首先被设置为 1，然后设为 $2 \cdot 1 = 2$ ，然后设为 $3 \cdot 2 = 6$ 。最后一次迭代中，这个值会乘以它自己，得到最后结果 36。对于 combine4 的情况来说，直到最后向量都保持不变，结束之前，最后一个元素会被设置为计算出来的值 $1 \cdot 2 \cdot 3 \cdot 5 = 30$ 。

当然，我们说明 combine3 和 combine4 之间差别的例子是人为设计的。有人会说 combine4 的行为更加符合函数描述的意图。不幸的是，编译器不能判断函数会在什么情况下被调用，以及程序员的本意可能是什么。取而代之，在编译 combine3 时，保守的方法是不断地读和写内存，即使这样做效率不太高。

 **练习题 5.4** 当用带命令行选项 “-O2” 的 GCC 来编译 combine3 时，得到的代码 CPE 性能远好于使用 -O1 时的：

函数	方法	整数		浮点数	
		+	*	+	*
combine3	用 -O1 编译	7.17	9.02	9.02	11.03
combine3	用 -O2 编译	1.60	3.01	3.01	5.01
combine4	累积在临时变量中	1.27	3.01	3.01	5.01

由此得到的性能与 combine4 相当，不过对于整数求和的情况除外，虽然性能已经得到了显著的提高，但还是低于 combine4。在检查编译器产生的汇编代码时，我们发现对内循环的一个有趣的变化：

```
Inner loop of combine3. data_t = double, OP = *. Compiled -O2
dest in %rbx, data+i in %rdx, data+length in %rax
Accumulated product in %xmm0
1 .L22:                                loop:
2 vmulsd (%rdx), %xmm0, %xmm0        Multiply product by data[i]
3 addq $8, %rdx                      Increment data+i
4 cmpq %rax, %rdx                    Compare to data+length
5 vmovsd %xmm0, (%rbx)               Store product at dest
6 jne .L22                           If !=, goto loop
```

把上面的代码与用优化等级 1 产生的代码进行比较：

```
Inner loop of combine3. data_t = double, OP = *. Compiled -O1
dest in %rbx, data+i in %rdx, data+length in %rax
1 .L17:                                loop:
2 vmovsd (%rbx), %xmm0               Read product from dest
3 vmulsd (%rdx), %xmm0, %xmm0        Multiply product by data[i]
4 vmovsd %xmm0, (%rbx)               Store product at dest

5 addq $8, %rdx                      Increment data+i
6 cmpq %rax, %rdx                    Compare to data+length
7 jne .L17                           If !=, goto loop
```

我们看到，除了指令顺序有些不同，唯一的区别就是使用更优化的版本不含有 vmovsd 指令，它实现的是从 dest 指定的位置读数据(第 2 行)。

- A. 寄存器 %xmm0 的角色在两个循环中有什么不同？
- B. 这个更优化的版本忠实地实现了 combine3 的 C 语言代码吗(包括在 dest 和向量数据之间使用内存别名的时候)？

C. 解释为什么这个优化保持了期望的行为, 或者给出一个例子说明它产生了与使用较少优化的代码不同的结果。

使用了这最后的变换, 至此, 对于每个元素的计算, 都只需要 1.25~5 个时钟周期。比起最开始采用优化时的 9~11 个周期, 这是相当大的提高了。现在我们想看看是什么因素在制约着代码的性能, 以及可以如何进一步提高。

5.7 理解现代处理器

到目前为止, 我们运用的优化都不依赖于目标机器的任何特性。这些优化只是简单地降低了过程调用的开销, 以及消除了一些重大的“妨碍优化的因素”, 这些因素会给优化编译器造成困难。随着试图进一步提高性能, 必须考虑利用处理器微体系结构的优化, 也就是处理器用来执行指令的底层系统设计。要想充分提高性能, 需要仔细分析程序, 同时代码的生成也要针对目标处理器进行调整。尽管如此, 我们还是能够运用一些基本的优化, 在很大一类处理器上产生整体的性能提高。我们在这里公布的详细性能结果, 对其他机器不一定有同样的效果, 但是操作和优化的通用原则对各种各样的机器都适用。

为了理解改进性能的方法, 我们需要理解现代处理器的微体系结构。由于大量的晶体管可以被集成到一块芯片上, 现代微处理器采用了复杂的硬件, 试图使程序性能最大化。带来的一个后果就是处理器的实际操作与通过观察机器级程序所察觉到的大相径庭。在代码级上, 看上去似乎是一次执行一条指令, 每条指令都包括从寄存器或内存取值, 执行一个操作, 并把结果存回到一个寄存器或内存位置。在实际的处理器中, 是同时对多条指令求值的, 这个现象称为指令级并行。在某些设计中, 可以有 100 或更多条指令在处理中。采用一些精细的机制来确保这种并行执行的行为, 正好能获得机器级程序要求的顺序语义模型的效果。现代微处理器取得的了不起的功绩之一是: 它们采用复杂而奇异的微处理器结构, 其中, 多条指令可以并行地执行, 同时又呈现出一种简单的顺序执行指令的表象。

虽然现代微处理器的详细设计超出了本书讲授的范围, 对这些微处理器运行的原则有一般性的了解就足够能够理解它们如何实现指令级并行。我们会发现两种下界描述了程序的最大性能。当一系列操作必须按照严格顺序执行时, 就会遇到延迟界限(latency bound), 因为在下一条指令开始之前, 这条指令必须结束。当代码中的数据相关限制了处理器利用指令级并行的能力时, 延迟界限能够限制程序性能。吞吐量界限(throughput bound)刻画了处理器功能单元的原始计算能力。这个界限是程序性能的终极限制。

5.7.1 整体操作

图 5-11 是现代微处理器的一个非常简单化的示意图。我们假想的处理器设计是不太严格地基于近期的 Intel 处理器的结构。这些处理器在工业界称为超标量(superscalar), 意思是它可以在每个时钟周期执行多个操作, 而且是乱序的(out-of-order), 意思就是指令执行的顺序不一定要与它们在机器级程序中的顺序一致。整个设计有两个主要部分: 指令控制单元(Instruction Control Unit, ICU)和执行单元(Execution Unit, EU)。前者负责从内存中读出指令序列, 并根据这些指令序列生成一组针对程序数据的基本操作; 而后者执行这些操作。和第 4 章中研究过的按序(in-order)流水线相比, 乱序处理器需要更大、更复杂的硬件, 但是它们能更好地达到更高的指令级并行度。